IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Final Project Report on

End to End Encryption for Supply Chain Cloud App

Student Name: T A b d u l S a m e e r

Register Number: 23BCCE135

Course: BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Industry Mentor: Mr. Shashank **Guided by:** Ms. Vijayalaxmi

Table of Contents

Section	Title	Page
	Executive Summary	1
1	**BACKGROUND**	2
1.1	Aim	2
1.2	Technologies	3-4
1.3	Hardware Architecture & Network Topology	4-5
1.4	Software Architecture & Framework	6-7
2	**SYSTEM**	8
2.1	Requirements Portfolio	8-15
2.2	Design Blueprint & Architecture	16-21
2.3	Implementation Details	22-25
2.4	Testing & Validation	26-28
2.5	Graphical User Interface (GUI)	29-30
2.6	Customer Testing	31
2.7	Evaluation	32-33
3	**PROJECT SNAPSHOTS (VISUAL EVIDENCE)**	34-39
4	**CONCLUSION**	40-41
5	**FUTURE DEVELOPMENT**	42-46
6	**REFERENCES**	47
7	**APPENDIX**	48
7.1	Sample Log Entries	48-49
7.2	Attack Graph Data	50
7.3	System Output Screens	51-52
7.4	Glossary of Terms	53-55

Executive Summary

The End-to-End Encryption for Supply Chain Cloud App project is developed to protect sensitive supply chain information while it is created, stored, transferred, and accessed through a cloud-based application. The project focuses on providing a secure communication environment for different supply chain stakeholders such as Admin, Manufacturer, Supplier, Warehouse, and Distributor.

The system uses Python and the Flask framework to create a lightweight web application. Sensitive files or messages are encrypted before storage using the Fernet encryption technique. The application stores only encrypted content and allows authorized users to retrieve readable information only through the decryption process.

A web-based dashboard is used to provide a simple interface for login, data upload, encrypted file access, and monitoring of user activity. The system also maintains audit records of important actions such as login, upload, file access, and decryption requests. This helps improve accountability and secure system operation.

The project is designed to be simple, practical, and suitable for academic demonstration. It combines core cybersecurity concepts such as confidentiality, secure storage, role-based access control, and protected communication in a cloud environment.

Overall, the project demonstrates how end-to-end encryption principles can be applied in a supply chain application to improve trust, privacy, and data security among connected business participants.

1. BACKGROUND

1. Aim

The primary aim of this project is to design and develop a secure cloud-based supply chain application that protects sensitive business information using end-to-end encryption principles. The system is intended to reduce the risk of unauthorized access by ensuring that important supply chain data is stored and shared in encrypted form.

The project focuses on creating a lightweight and practical encryption-based solution using Python, Flask, HTML, and CSS. It aims to protect files and messages exchanged between supply chain participants and to allow access only to authorized users.

Another important objective is to provide a simple user interface through which users can upload files, retrieve encrypted records, and decrypt content securely. The project also aims to demonstrate role-based access, secure data handling, and audit monitoring in an educational environment.

Overall, the purpose of this project is to build a simple but effective security-oriented cloud application that demonstrates how encrypted communication can improve trust and confidentiality in supply chain systems.

1.2 Technologies

The development of the End-to-End Encryption for Supply Chain Cloud App uses a combination of technologies that work together to provide secure data handling and a simple web interface.

Python is the main programming language used in the project. It is used to implement backend logic, encryption handling, file management, and user request processing. Its readability and strong library support make it suitable for developing a secure academic application.

Flask is used as the web framework to build the application backend. It manages routing, user interaction, request handling, and communication between the user interface and the encryption modules.

HTML and CSS are used to design the frontend interface. These technologies help create pages such as login, dashboard, upload, and file-view interfaces in a clean and understandable format.

The Cryptography library, especially the Fernet module, is used to implement encryption and decryption. Fernet helps convert readable data into ciphertext and allows safe recovery of the original data only through the correct process.

Simple file-based storage is used for keeping encrypted data and related records. This approach keeps the project lightweight and easy to implement for demonstration purposes.

Overall, these technologies provide a practical foundation for building a secure supply chain cloud application.

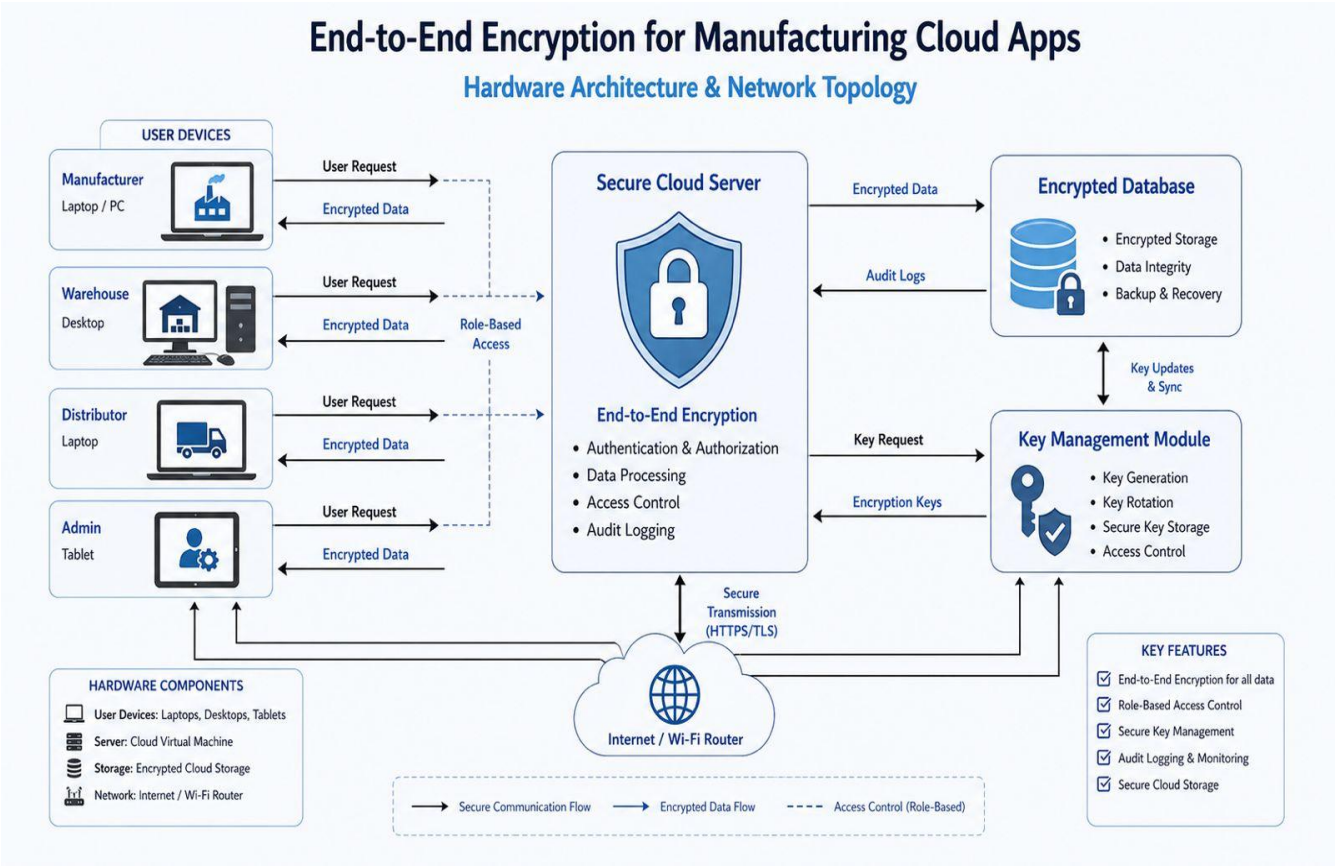
1.3 Hardware Architecture & Network Topology

The hardware architecture of the system is simple and suitable for a student project environment. The application primarily requires a laptop or desktop computer to run the Flask server and manage encryption operations.

The server system acts as the main processing unit where user requests are handled, files are encrypted, and encrypted records are stored. A web browser is used on the same machine or other devices to access the application interface.

Multiple users can access the application through devices connected to the same local network or a controlled cloud-access environment. These users may represent roles such as Admin, Manufacturer, Supplier, Warehouse, or Distributor.

A basic network setup using Wi-Fi or LAN is sufficient for demonstrating how users interact with the application.
The architecture remains cost-effective and does not require specialized hardware.



Overall, the hardware arrangement is lightweight, affordable, and suitable for practical academic testing.

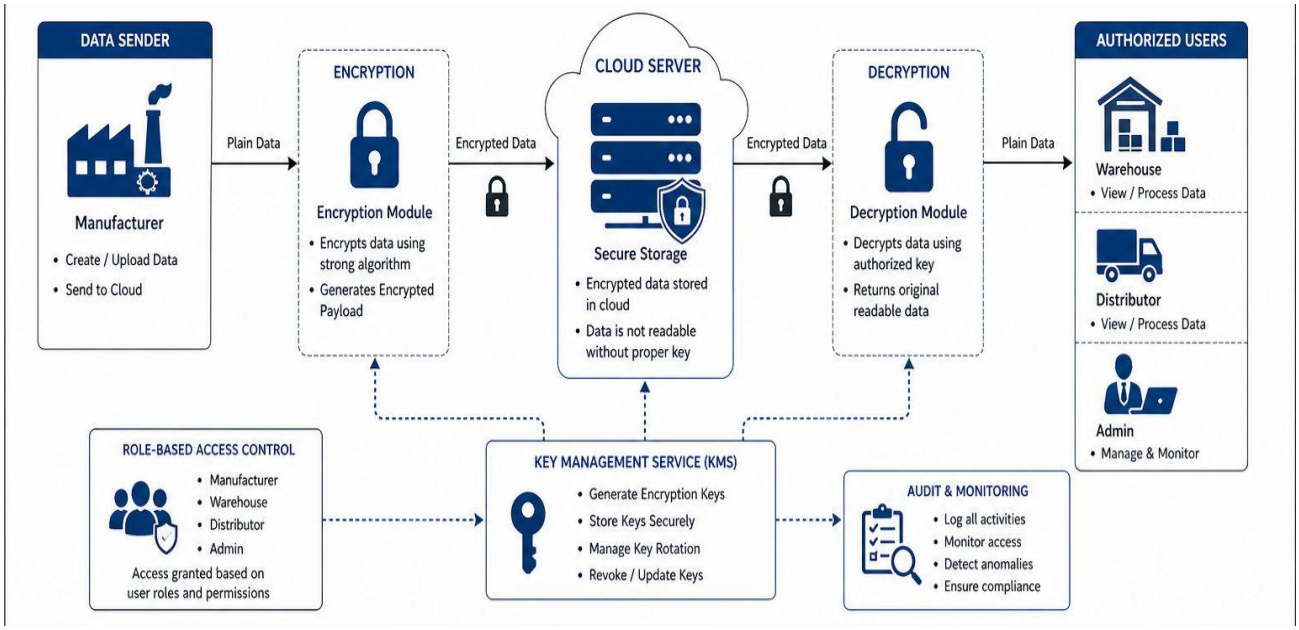
1.4 Software Architecture & Framework

The software architecture of the End-to-End Encryption for Supply Chain Cloud App is designed using a modular structure, where each part of the system performs a separate but connected function.

The frontend layer provides user-facing pages such as login, dashboard, upload page, encrypted records page, and access result page. The backend layer, implemented in Flask, handles routing, authentication checks, role-based access, and secure processing of uploaded data.

The encryption module is responsible for converting plain text or uploaded file content into encrypted form before storage. The decryption module retrieves the encrypted content and restores readable data only for approved requests. A storage layer maintains encrypted records, while an audit layer records system activities.

This modular structure improves clarity, maintenance, and future scalability. It also ensures that the system can be extended later with cloud storage, stronger authentication, or database integration.



This diagram illustrates the low-level design of an end-to-end encryption system for manufacturing cloud applications, where data generated by the manufacturer is first encrypted before being transmitted to the cloud server for secure storage. The encrypted data remains unreadable within the cloud and can only be accessed by authorized users such as warehouse staff, distributors, or administrators through a controlled decryption process. The system incorporates role-based access control to ensure that users can only access permitted data, while a key management service securely handles encryption keys. Additionally, audit and monitoring mechanisms track system activities, ensuring data confidentiality, integrity, and secure communication throughout the entire workflow..

2. SYSTEM

2.1 Requirements Portfolio

The requirements portfolio of the project defines the main features and operating conditions needed for the system to function effectively. These requirements ensure that the application protects data properly while remaining easy to use.

The system should allow users to log in and access features according to their assigned roles. It should support encrypted file or message submission, secure storage, controlled retrieval, and decryption of authorized content. It should also record important user activities for audit purposes.

The application should provide a simple dashboard, smooth page navigation, and clear display of upload and access results. It should be lightweight and able to run on standard systems without requiring advanced infrastructure.

In addition, the system should be easy to deploy and maintain, making it suitable for academic use and small-scale testing.

Table: System Requirements Overview

Category	Requirement	Description
Functional	User login	Authentication users before access
Functional	Data Encryption	Encrypts files or messages before storage
Functional	Secure Storage	Stores only protected data
User	Dashboard UI	Displays logs and alerts
User	Navigation	Easy switching between pages
Environment	Platform Support	Runs on Windows/Linux
Environment	Browser Access	Supports modern browsers

The following table provides a summary of the key system requirements of the Honeypot Network for Retail Threat Intelligence system. It categorizes the requirements into functional, user, and environmental aspects, highlighting the essential features and conditions needed for the system to operate effectively. This structured representation helps in understanding the core capabilities and expected behavior of the system.

2.1.1 Functional Requirement

Functional requirements describe the main actions that the system must perform. The application should authenticate users before giving access to protected functions. It should allow authorized users to upload supply chain files or enter sensitive information that needs protection. Before storage, the system must encrypt the content using Fernet-based encryption.

The application should store encrypted content securely and retrieve it only when a valid user requests access. Decryption should occur only after authorization checks are completed. The system should also record uploads, access requests, decryption actions, and logout events.

The system should also allow authorized users to view stored records in encrypted form and request decryption when required. Decryption should only be performed after proper authorization checks, ensuring that only permitted users can access the original data. The application must support smooth navigation between different sections such as dashboard, upload page, file list, and result display. Additionally, the system should maintain an audit log to record important actions such as login attempts, file uploads, encryption, and decryption activities.

All data (orders, invoices, shipment details) must be encrypted before sending and decrypted only by the intended receiver.

- Use encryption algorithms like:
- AES (for data encryption)
- RSA (for key exchange)
- Even the cloud server should not read the data (zero-knowledge concept)

Overall, the functional requirements ensure secure storage, controlled access, and protected data sharing.

2.1.2 User Requirements

User requirements define how the system should interact with the people using it.

The application should provide a simple and understandable interface so that users can log in, upload data, and retrieve files without difficulty. The dashboard should display essential options clearly and avoid unnecessary complexity.

Users should be able to navigate smoothly between login, dashboard, upload, encrypted files, and output pages. The system should also display confirmations or warnings clearly so that users can understand whether a request is successful or denied.

. Users require the ability to register and log in securely, manage their profiles, and perform supply chain operations like creating orders, tracking shipments, and sharing documents.

A key requirement is that all sensitive data, including transaction details and documents, must remain confidential through end-to-end encryption, ensuring that only authorized users can access it. Additionally, users expect role-based access so that each participant can view only relevant information. The system should also provide real-time updates, notifications, and an easy-to-use interface, while maintaining high security, data privacy, and reliability throughout the entire process.

Overall, the system should provide ease of use, clarity, and smooth interaction.
.

2.1.3 Environmental Requirements

Environmental requirements define the setup in which the application should operate effectively.

The system should run on a standard laptop or desktop computer with basic processing capability. It should support common operating systems such as Windows or Linux. A browser is required to access the Flask-based interface.

The backend should support Python and Flask, while storage may use files or lightweight local records. The application should remain efficient without high memory or processing demands.

It requires a stable internet connection for secure data transmission and real-time updates. On the server side, the system should operate on a reliable cloud platform with support for encryption technologies and secure storage. The development environment may include tools such as Python, Visual Studio Code, and relevant libraries for implementing encryption. Additionally, the system should be compatible with common operating systems like Windows, Linux, and Android, ensuring flexibility and ease of deployment in different organizational setups.

the system environment should ensure basic security configurations such as firewall protection, secure network access, and updated software dependencies to prevent vulnerabilities. The application should be able to run in a local development environment as well as a cloud-based environment without major modifications. It should also support scalability to handle increased data or user activity if required in future enhancements. Proper configuration of Python libraries and environment variables is necessary to maintain smooth execution of encryption functionalities.

.

Table: Environmental Requirements Summary:

Component	Requirement
Hardware	Laptop or desktop system
Operating System	Windows / Linux
Network	Cryptography library (Fernet)
Backend	Python + Flask
Frontend	Web browser (Chrome, Edge)
Storage	JSON file
Resources	Low memory and CPU usage

2.2 Design Blueprint & Architecture

The design blueprint of the project follows a secure modular architecture. The system is divided into user interface, authentication, encryption, storage, decryption, and audit monitoring layers.

The process begins when a user logs into the system and submits data through the interface. The backend verifies the request, sends the content to the encryption module, and stores the encrypted result in the storage layer.

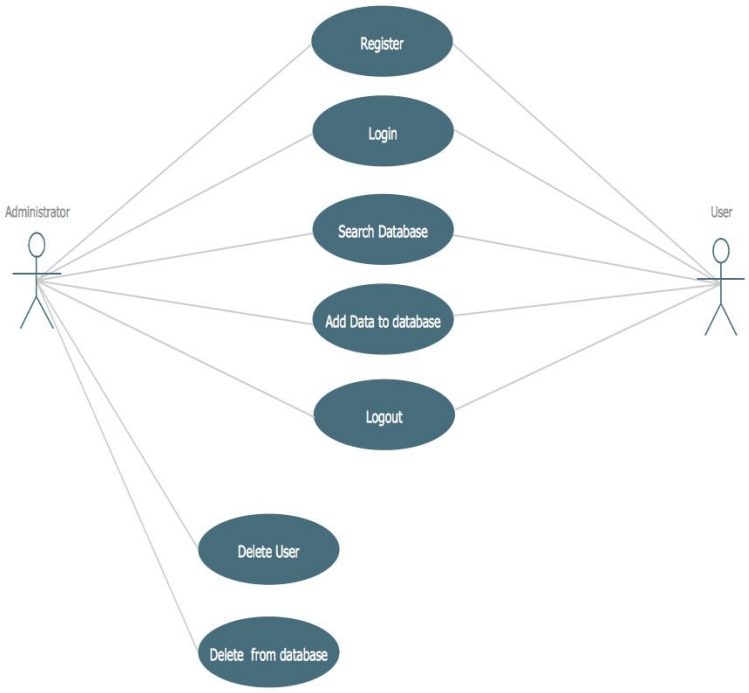
When a valid user later requests access, the backend checks role permissions, retrieves the encrypted content, and forwards it to the decryption module. The readable output is then shown to the user. At every major stage, the audit log records the activity for monitoring and accountability.

.

The system architecture follows a client-server model integrated with end-to-end encryption mechanisms. Users interact with the application through a frontend interface, while the backend server manages requests, authentication, and encrypted data storage on the cloud. Before data is transmitted, it is encrypted using secure algorithms, and only authorized users with the correct keys can decrypt it. The cloud server acts as a storage and communication medium but cannot access the actual content of the data. The architecture includes key components such as user devices, application server, encryption/decryption module, and cloud database, all connected through secure communication protocols. This layered architecture ensures high security, efficient data handling, and reliable system performance.

This modular design ensures that each component works independently while contributing to the overall functionality of the system. It also allows easy scalability, where additional features such as database integration or advanced analytics can be added in the future.

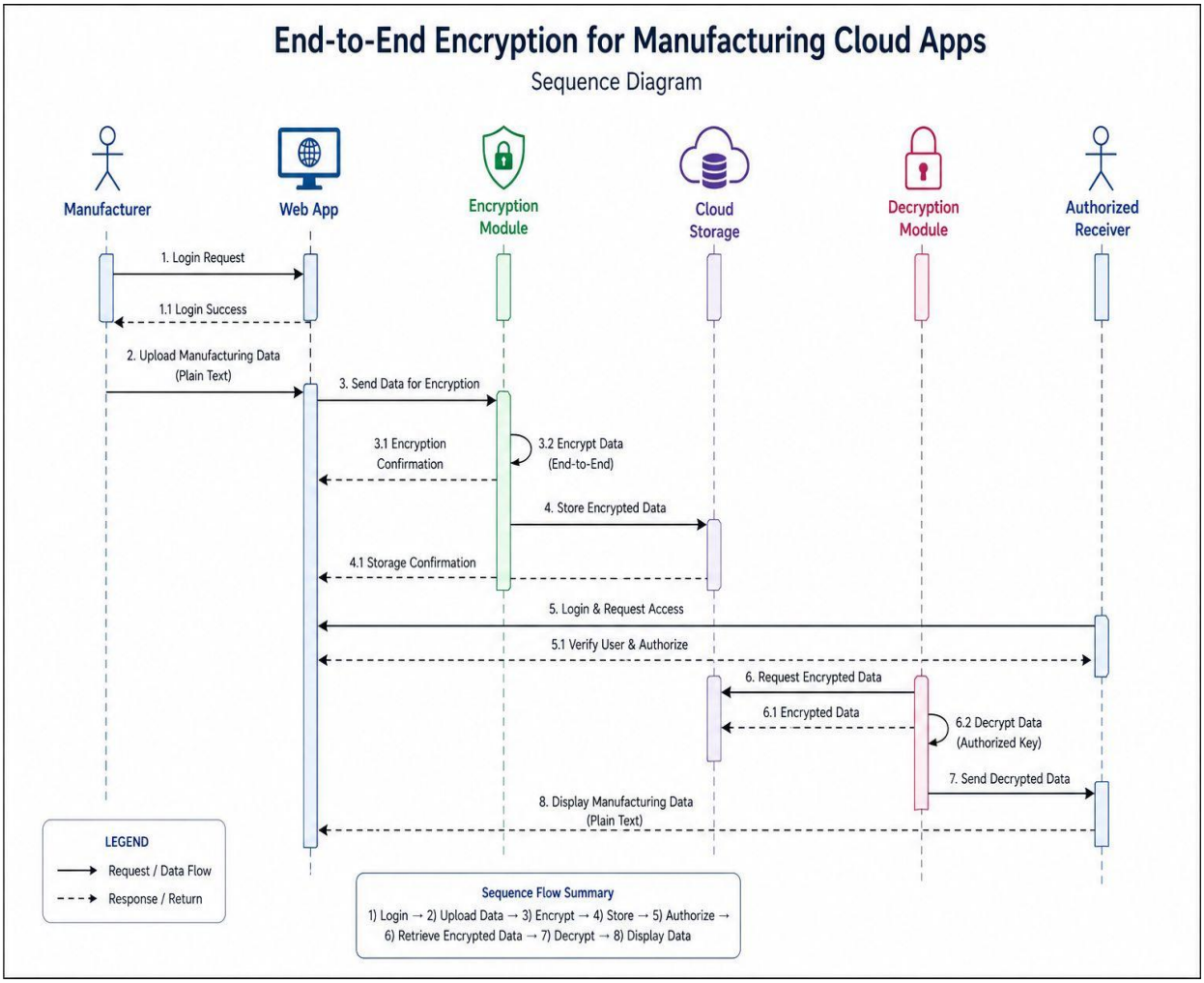
2.2.1 Use Case Diagram



The main actors in the system are Admin, Manufacturer, Supplier, Warehouse, and Distributor. The Admin supervises users and audit records. Other roles interact with the application according to their responsibilities in the supply chain workflow.

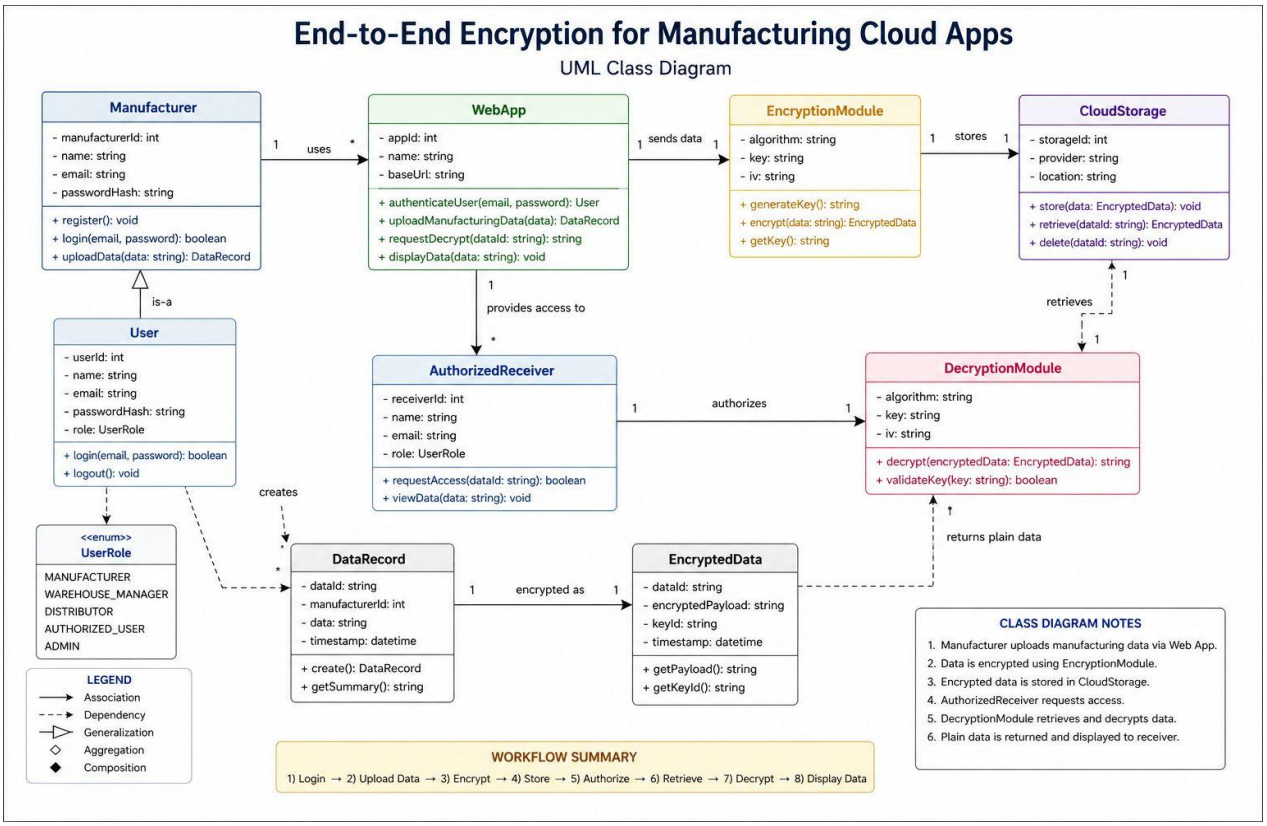
The major use cases include login, upload encrypted data, view stored records, request decrypted output, and view logs. These use cases together describe a secure supply chain cloud workflow.

2.2.2 Sequence Diagram



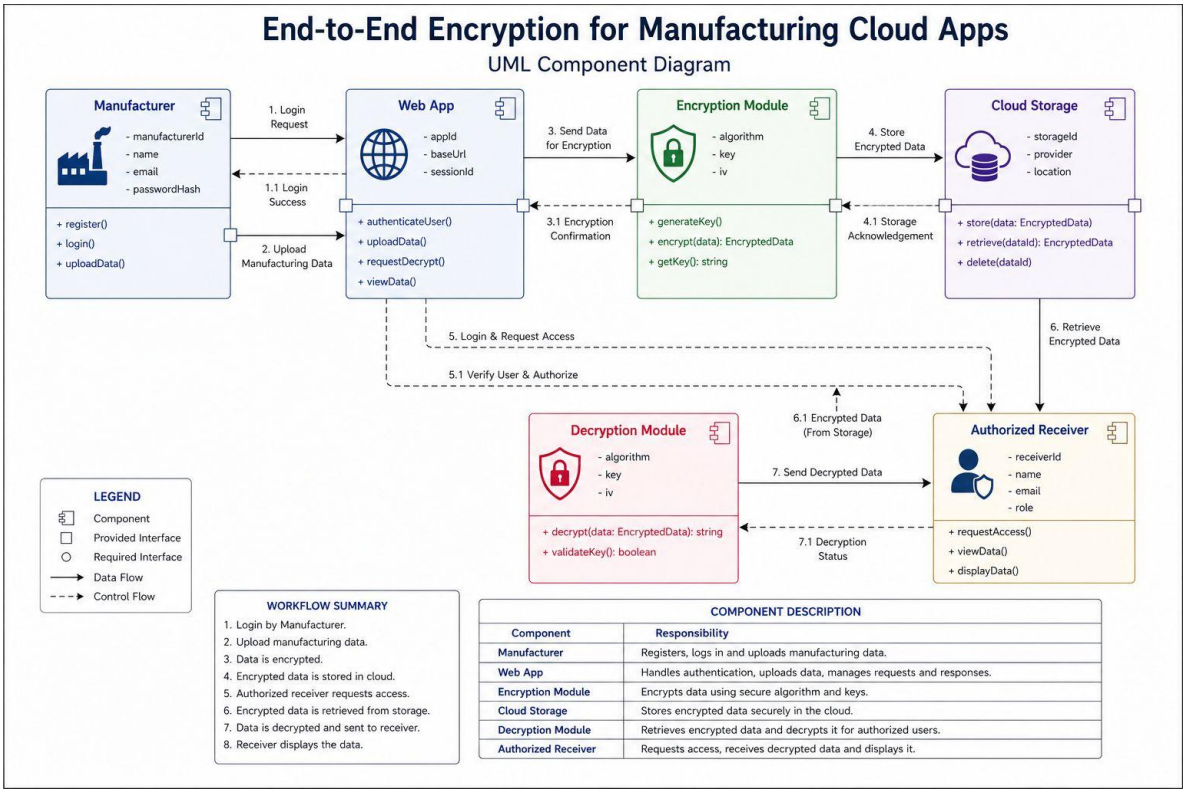
The sequence begins when the user logs into the application. After successful login, the user uploads a file or enters sensitive data. The backend sends the content to the encryption module and receives the encrypted result. The encrypted data is stored securely. When the user requests the stored content later, the backend checks authorization and calls the decryption module before showing the original result.

2.2.3 Class diagram



The class-level structure may include classes such as User, AuthManager, EncryptionService, StorageManager, AuditLogger, and FileRecord. These classes work together to manage user sessions, encrypt and decrypt records, and track system activity.

2.2.4 Component Design



This component design diagram illustrates the architecture of an end-to-end encryption system for manufacturing cloud applications, highlighting how different modules interact to ensure secure data handling. The manufacturer first logs into the web application and uploads manufacturing data, which is then passed to the encryption module where it is securely encrypted using cryptographic algorithms and keys. The encrypted data is stored in cloud storage, ensuring that sensitive information remains protected at rest. When an authorized receiver requests access, the system verifies user permissions and retrieves the encrypted data, which is then processed by the decryption module using valid keys. Finally, the decrypted data is delivered to the authorized user through the web application interface. This design ensures confidentiality, secure storage, controlled access, and smooth data flow across all system components.

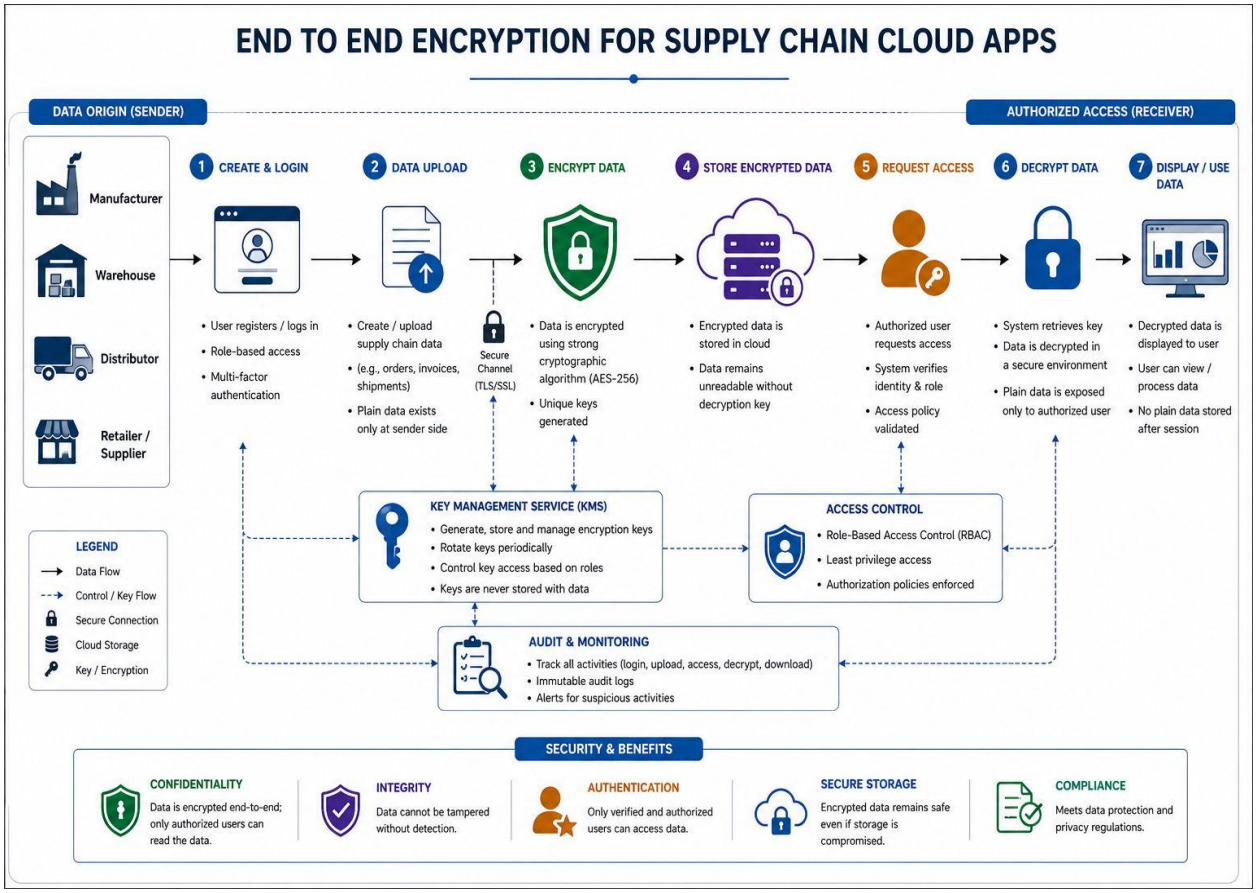
2.3 Implementation Details

The implementation of the project is carried out using Python, Flask, HTML, CSS, and the Cryptography library. The system is designed to be simple, secure, and easy to demonstrate.

User requests are received through the Flask routes. After login validation, the backend accepts file uploads or text input and passes them to the encryption logic. The Fernet module encrypts the content and returns the protected result. The encrypted content is then written to storage.

When an authorized request is made to access stored data, the application retrieves the encrypted content, performs role verification, and sends the data through the decryption process. After successful decryption, the readable output is displayed on the interface.

The system also maintains audit records of important user activities. This helps monitor the application and supports accountability.



This flow diagram illustrates the complete process of end-to-end encryption in a supply chain cloud application, ensuring secure data handling from sender to receiver. Initially, users such as manufacturers, warehouses, distributors, or retailers log into the system and upload supply chain data. The data is then encrypted using strong encryption algorithms before being transmitted and stored securely in the cloud, making it unreadable to unauthorized users. A key management system handles encryption keys separately, enhancing security, while access control mechanisms verify user roles and permissions. When an authorized user requests access, the system retrieves the encrypted data, decrypts it using valid keys, and displays the original information. Additionally, audit and monitoring features track all activities, ensuring transparency and security. This process guarantees data confidentiality, integrity, and secure communication across the entire supply chain system.

2.4 Testing & Validation

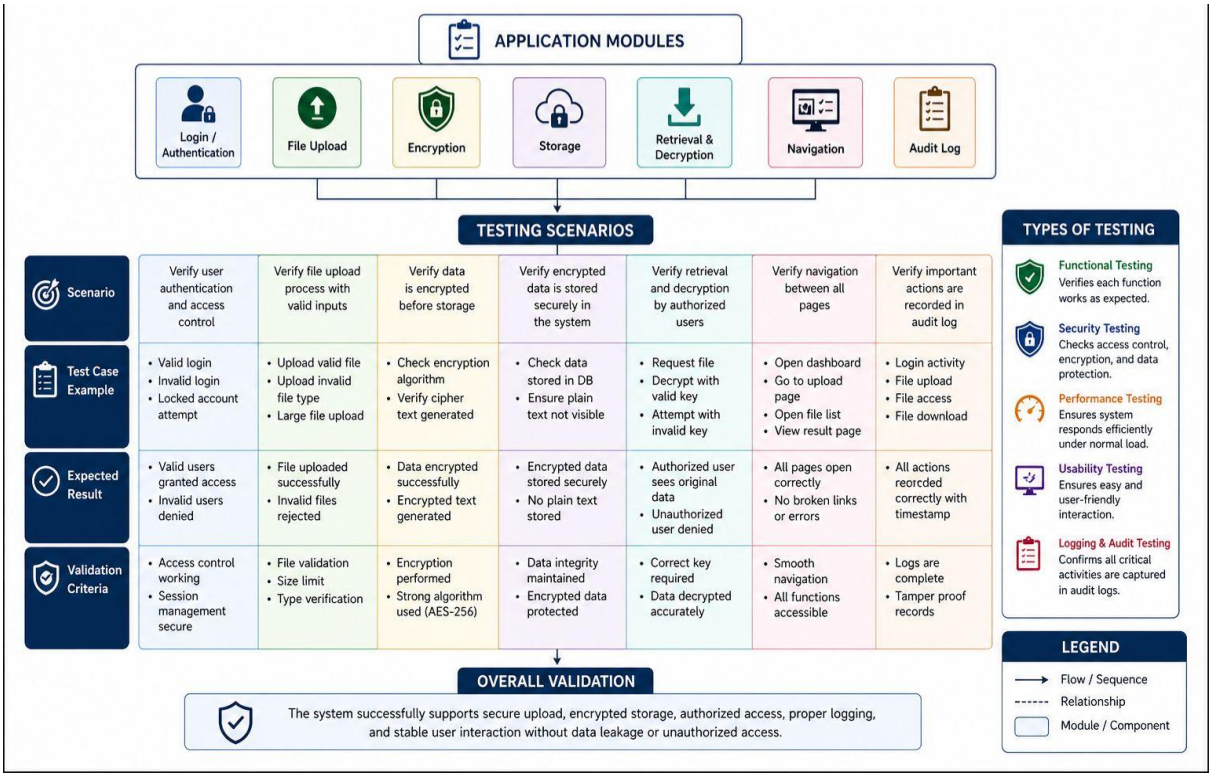
Testing and validation are performed to confirm that the application functions correctly and securely.

The login process is tested to ensure that valid users are granted access and invalid users are denied. File upload and encryption are tested to confirm that submitted content is properly converted into encrypted form before storage. Retrieval and decryption are tested to confirm that readable content is shown only to authorized users.

Navigation between dashboard, upload, file list, and result pages is also verified. Additional tests are carried out to check whether the audit log records important actions correctly.

Overall, the testing confirms that the application supports secure upload, protected storage, authorized access, and stable user interaction

Furthermore, performance testing is conducted to ensure that the system responds efficiently under normal usage conditions without significant delays during encryption and decryption processes. Security testing is also performed to identify potential vulnerabilities such as unauthorized access attempts, data leakage, or improper key handling. Input validation checks are implemented to prevent incorrect or malicious data from being processed by the system. In addition, usability testing is carried out to ensure that the user interface is simple, clear, and easy to navigate for all types of users. Error handling is verified to confirm that the system provides appropriate messages and does not crash during unexpected situations. These combined testing and validation procedures ensure that the application is reliable, secure, and ready for practical use in a real-world environment.



This diagram represents the testing and validation scenarios for the end-to-end encryption system in a manufacturing cloud application. It highlights key application modules such as login and authentication, file upload, encryption, storage, retrieval and decryption, navigation, and audit logging. Each module is tested through specific scenarios to ensure correct functionality, including validating user access control, verifying file upload with proper inputs, confirming that data is encrypted before storage, and ensuring that encrypted data is securely stored without exposing plain text. Retrieval and decryption processes are tested to confirm that only authorized users can access and view the original data using valid keys. Navigation testing ensures smooth movement across different pages of the application, while audit log testing verifies that all important user actions are accurately recorded.

2.5 Graphical User Interface (GUI) Layout

The graphical user interface of the project is designed to be simple, clean, and easy to use. It helps users interact with secure features without needing complex technical knowledge.

The dashboard acts as the central page of the system and provides quick access to upload, view files, decrypt data, and check status. The upload page allows users to select files or enter text, while the files page lists protected records available for authorized access.

The result page displays decrypted output only after successful verification. The audit page, where applicable, allows the Admin to view key activity records. The interface emphasizes clarity, usability, and secure workflow.

Furthermore, the GUI is designed with a structured layout that ensures all elements are properly aligned and easy to navigate. Clear labels, buttons, and input fields guide the user through each step of the process, reducing confusion and improving efficiency. Basic validation messages are provided to inform users about successful operations or errors, such as invalid input or unauthorized access attempts. The interface also ensures that sensitive information is not directly visible unless proper decryption is performed, thereby maintaining data confidentiality. Consistent design elements such as colors, fonts, and spacing are used throughout the application to provide a professional and uniform appearance. Overall, the GUI enhances user experience by combining simplicity, security, and functionality in a well-organized manner.

Table: GUI Components Overview:

Component	Description
Dashboard	Displays main navigation and summary
Login page	Authenticates users
Alerts Page	Highlights suspicious activities
Navigation Menu	Provides easy page access
Audit Page	Shows logged system activity
Result page	Displays decrypted output

The following table presents an overview of the main graphical user interface (GUI) components of the Supply chain system. Each component plays a specific role in displaying information, supporting navigation, and enhancing user interaction for effective monitoring of network activity.

The graphical user interface of the system is illustrated in Section 3 (Project Snapshots), where the dashboard, logs, and alerts views are presented in detail.

2.6 User Testing

User testing is conducted to evaluate how effectively the application works in a practical environment. Test users are allowed to log in, upload data, view encrypted records, and retrieve authorized output.

2.7 Evaluation

The evaluation of the End-to-End Encryption for Supply Chain Cloud App is conducted to assess functionality, security, usability, and overall effectiveness.

The system successfully encrypts supply chain data before storage and supports controlled decryption for authorized users. It provides a simple dashboard interface, smooth navigation, and effective integration of backend logic with encryption functions.

From a security perspective, the project demonstrates confidentiality, protected storage, role-based access, and audit logging. From a usability perspective, the interface remains easy to understand and practical for project demonstration.

However, the current version also has limitations. It uses simple file-based storage, basic session handling, and academic-scale encryption workflow. Advanced enterprise-level features such as distributed key management, cloud-native deployment, and high-scale collaboration are not included.

The modular design of the application also enhances maintainability and scalability. Each component, such as the user interface, encryption module, and storage mechanism, operates independently, allowing future improvements or upgrades without affecting the entire system. This flexibility makes it easier to extend the project by integrating additional features such as database support, user authentication systems, or cloud deployment.

Furthermore, the project provides a practical demonstration of how encryption techniques can be applied in real-world scenarios. It helps in understanding the importance of secure data transmission and storage, especially in supply chain environments where sensitive information is frequently exchanged between multiple stakeholders.

For future enhancement, the system can be improved by implementing advanced security features such as multi-factor authentication, secure key exchange mechanisms, and integration with cloud platforms like AWS or Azure.Overall, the evaluation confirms that the project not only achieves its intended goals but also serves as a valuable learning model for secure application development, demonstrating both technical implementation and practical relevance in the field of cloud-based systems.

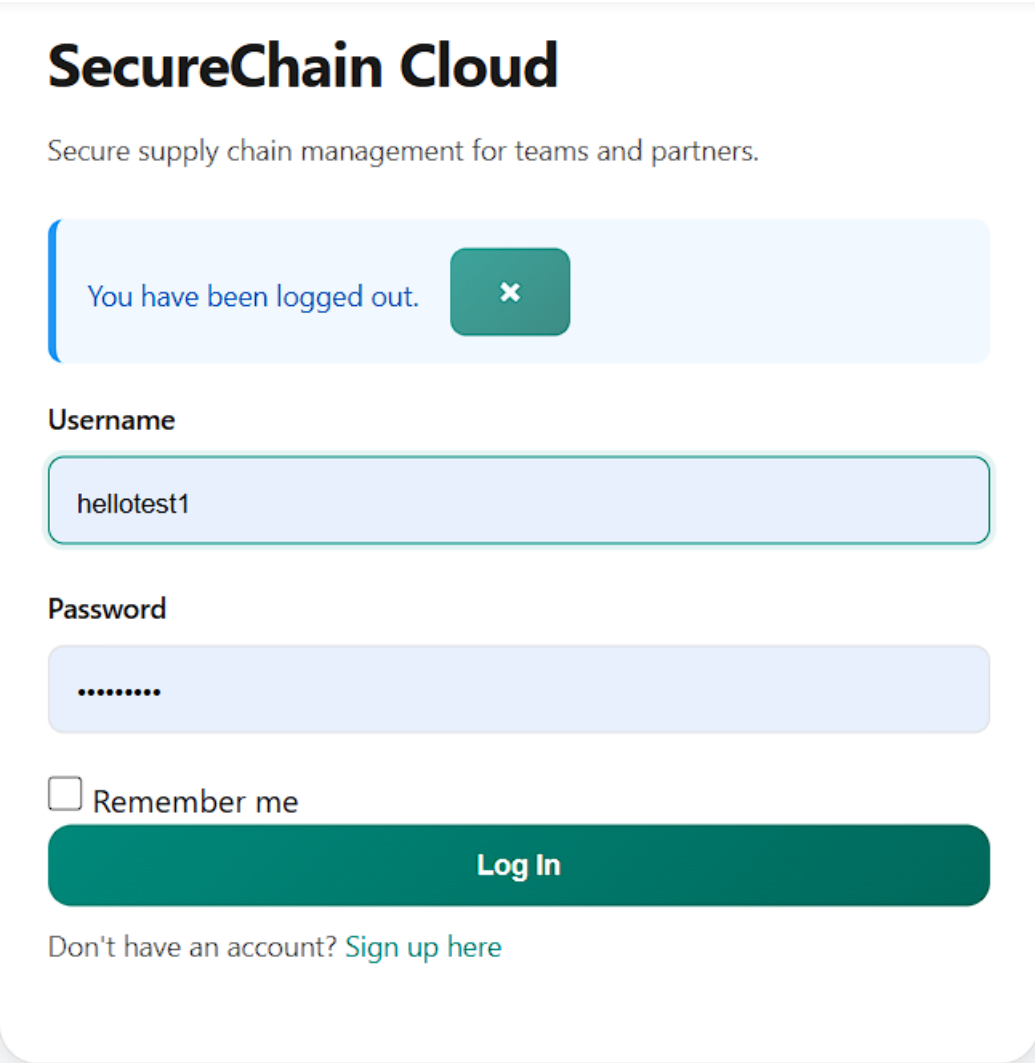
Table: System Evaluation Summary

Criteria	Evaluation
Functionality	Successfully encrypts, stores, and retrieves data
Security	Protects data through encryption and access control
Performance	Lightweight and responsive
Usability	Easy to use interface
Scalability	Limited in current academic version

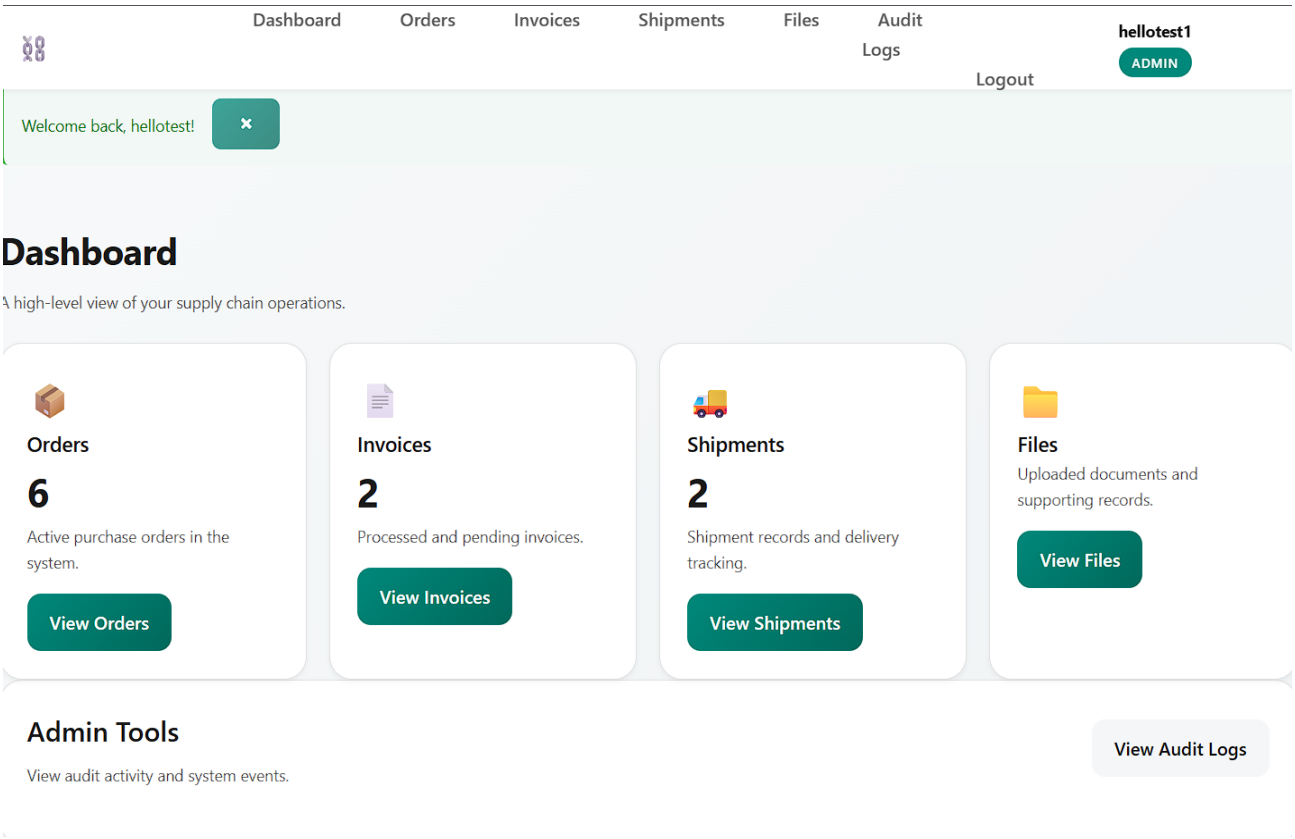
3. PROJECT SNAPSHOTS (VISUAL EVIDENCE)

This section presents the expected visual outputs of the End-to-End Encryption for Supply Chain Cloud App. It may include screenshots of the login page, dashboard, upload page, encrypted records page, and decrypted output page.

1. Login View

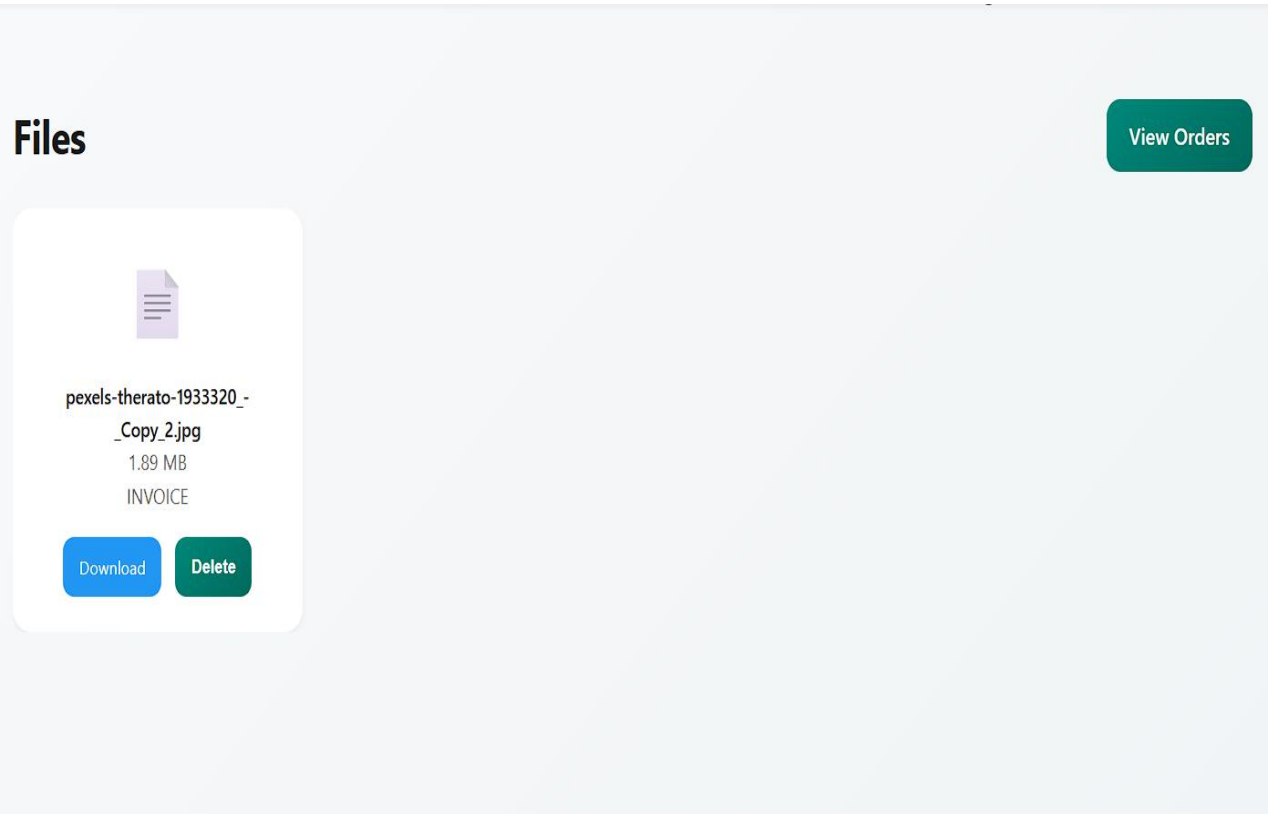


3.2 Dashboard View



The dashboard provides the main navigation options for secure upload, encrypted file access, and activity monitoring. It acts as the central control page of the application.

3.3 Upload Interface



The upload interface allows authorized users to submit files or messages for encryption. After submission, the data is processed securely and stored in encrypted form.

3.4 Stored Records View

Schema | Query | Data | ER Diagram

Search table page... Read-only

CID	NAME	TYPE	N
0	id	INTEGER	1
1	username	VARCHAR(80)	1
2	email	VARCHAR(120)	1
3	password_hash	VARCHAR(255)	1
4	full_name	VARCHAR(120)	1
5	role	VARCHAR(20)	1
6	company_name	VARCHAR(120)	0
7	is_active	BOOLEAN	0
8	created_at	DATETIME	0
9	updated_at	DATETIME	0

The stored records page displays available encrypted files or records. Users can select items for access based on permissions.

3.6 Audit Activity View

System Audit Logs

All system activities and changes recorded for compliance and tracking.

Timestamp	User	Action	Resource Type	Resource ID	Details
2026-04-27 19:48:49	hellotest	ACCESSED	Shipment	1	▶ Show
2026-04-27 19:48:37	hellotest	DOWNLOADED	File	1	▶ Show
2026-04-27 19:24:20	hellotest	ACCESSED	Invoice	1	▶ Show
2026-04-27 14:39:43	hellotest1	ACCESSED	Order	2	▶ Show
2026-04-27 14:34:46	qwert	ACCESSED	Order	2	▶ Show
2026-04-27 14:34:29	hellotest1	MESSAGED	Order	2	▶ Show
2026-04-27 14:34:22	hellotest1	ACCESSED	Order	2	▶ Show

The audit activity page displays important system events such as login, upload, and decrypt actions.
This improves monitoring and accountability..

4. CONCLUSION

The End-to-End Encryption for Supply Chain Cloud App successfully demonstrates a secure approach to handling sensitive business information in a cloud-oriented environment. The system protects data by encrypting it before storage and allowing access only through controlled decryption.

The implementation using Python, Flask, and Fernet encryption provides a practical and understandable model for academic use. The project shows how confidentiality, role-based access, secure storage, and audit monitoring can be combined into a simple but effective application.

The system supports secure file submission, protected retrieval, and user-friendly interaction. It also highlights the importance of applying encryption-based protection in supply chain communication, where multiple stakeholders may access important data.

Although the current version is lightweight, it provides a strong base for future enhancement. Overall, the project meets its objective of demonstrating a secure cloud application for protected supply chain data sharing.

In addition, the project helps in understanding how encryption can improve trust between different supply chain participants. It also shows how secure design principles can be applied in real application workflows rather than only in theory.

5. FUTURE SCOPE

The current project provides a strong academic foundation, but there are several areas for future enhancement.

One important improvement is the use of database systems such as MySQL or MongoDB instead of simple file-based storage. This would support larger datasets, better indexing, and easier record management.

Another improvement is stronger authentication, including multi-factor login and more advanced role handling. This would make the system more suitable for real-world cloud deployment.

The project can also be improved by integrating secure cloud storage services, automated key rotation, and structured key management. These features would make the encryption workflow more robust and enterprise-oriented.

Real-time notifications for file access or suspicious attempts can also be added. Interactive dashboards, advanced filtering, and reporting features would improve usability and monitoring.

The application may also be expanded to support multiple organizations, distributed users, and stronger audit analysis. With these enhancements, the system can move from an academic prototype toward a more practical secure supply chain platform.

The system can also be extended to support multiple honeypot nodes deployed across different network environments. This would allow collection of distributed attack data, improving overall threat intelligence and providing a broader view of attacker behavior.

Another improvement is the addition of authentication and role-based access control. This would ensure that only authorized users can access the dashboard and sensitive data, improving system security.

Finally, the system can be integrated with other security tools such as intrusion detection systems (IDS) or firewalls. This would allow automated responses to detected threats, making the system more proactive and effective in preventing attacks.
Overall, these enhancements would transform the honeypot system into a scalable, intelligent, and enterprise-ready security monitoring solution capable of handling real-world cybersecurity challenges.

Another important enhancement is the integration of automated response mechanisms. The system can be extended to block suspicious IP addresses or trigger firewall rules when repeated attack patterns are detected. This would make the system more proactive rather than just monitoring.

The implementation of real-time dashboards with live data streaming can also improve the system. Technologies such as WebSockets can be used to provide instant updates without refreshing the page.

The system can also be extended to support different types of honeypots, such as web-based or application-level honeypots, to capture more diverse attack patterns. This would increase the effectiveness of the system in real-world scenarios.

Integration with centralized logging and monitoring tools such as ELK Stack (Elasticsearch, Logstash, Kibana) can provide advanced visualization and large-scale data analysis capabilities.

6. REFERENCES

1. Python Software Foundation, Python Documentation. Available: https://docs.python.org/

2. Pallets Projects, Flask Web Framework Documentation. Available: https://flask.palletsprojects.com/

3. Cryptography Library Documentation. Available: https://cryptography.io/

4. OWASP Cryptographic Storage Cheat Sheet. Available: https://cheatsheetseries.owasp.org/

5. NIST Cybersecurity Publications. Available: https://www.nist.gov/

6. End-to-End Encryption Concept Overview. Available: https://en.wikipedia.org/wiki/End-to-end_encryption

7. APPENDIX

This section presents sample records handled by the system during testing. The content shown below is illustrative and demonstrates how encrypted storage and access tracking may be represented.

Timestamp	User Role	File Name	Status	Action
2026-04-20 10:15:21	Manufacturer	invoice_01.pdf	Success	Uploaded & Encrypted
2026-04-20 10:20:42	Warehouse	Stock_update.txt	Success	Access Requested
2026-04-20 10:22:03	Warehouse	Stock_update.txt	Success	Decrypted
2026-04-20 10:30:09	Supplier	dispatch_note.pdf	Success	Upload & Encrypted
2026-04-20 10:35:11	Distributor	route_plan.txt	Denied	Unauthorized Access
2026-04-20 10:40:10	Supplier	dispatch_note.pdf	Success	Upload & Encrypted

An alert status indicates suspicious behavior where the number of attempts exceeds the predefined threshold.

By analyzing these log entries, users can identify patterns in attacker behavior, such as repeated access attempts from the same IP address. This helps in understanding potential threats and improving network security monitoring.

The structured format of the logs ensures that data is easy to read, manage, and analyze, making the system effective for both monitoring and learning purposes.

7.2 Access Summary Data

This section provides a simple dataset representing how user actions may be summarized during system use.

User Role	Number of Uploads	Number of Access Requests	Status
Admin	2	5	Active
Manufacturer	4	3	Active
Supplier	3	2	Active
Distributor	2	3	Restricted on some records

7.4 Glossary of Terms

This section explains important technical terms used in the project.

Term	Description
End to End Encryption	A method of protecting data so only intended users can read it
Ciphertext	Encrypted form of readable data
Plain Text	Original readable content before encryption
Fernet	A symmetric encryption method used in the project
Flask	Python web framework used for the application backend
Audit Log	Record of important user or system actions
Role Based Access	Access control based on assigned user role
Secure Storage	Storage of protected encrypted records
Decryption	Process of converting ciphertext back to readable form
Supply Chain	Network of entities involved in production and delivery